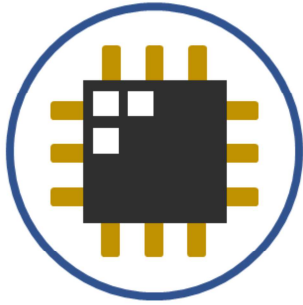
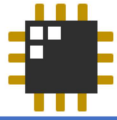




EEE 143: Switching Theory and Digital Logic Design

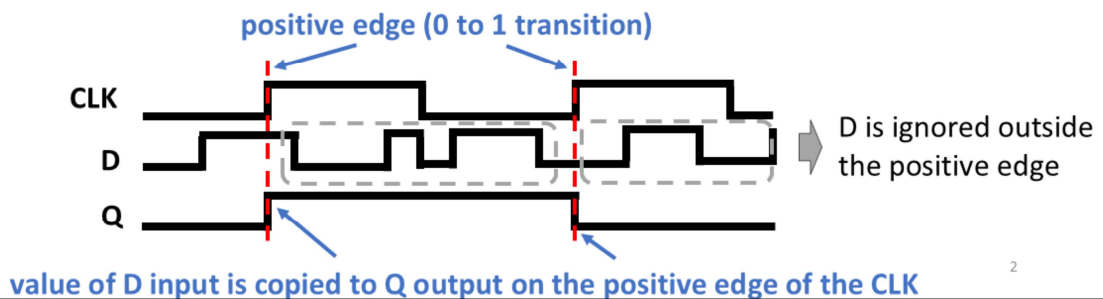
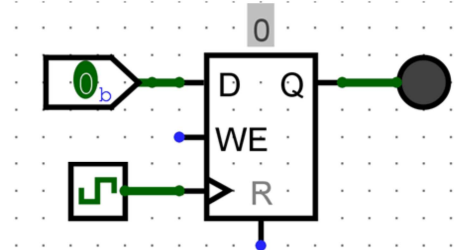
Lecture 9: Analysis of Synchronous Sequential Circuits

1

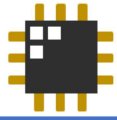


Last lecture

- Flipflops: A logic element that **stores** state
 - Synchronous to the clock signal
 - Holds a 0 or 1 at the output **Q** independent of the **D** input and is **triggered** to update at every rising/positive edge of the clock (**CLK**) input.

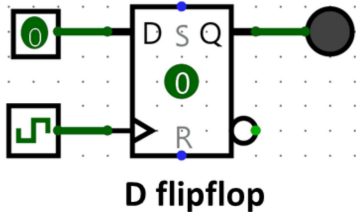


Just a quick review of what we covered about flipflops last lecture. We know that flipflop is a logic element that stores state. They hold a value of either 0 or 1 and update their state on the rising edge of the clock signal. They also help in synchronizing data across a system, making sure everything is updated at the right time.

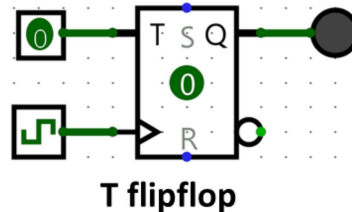


Last lecture

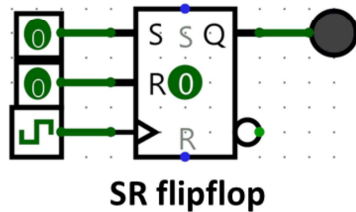
- Different flipflop flavors



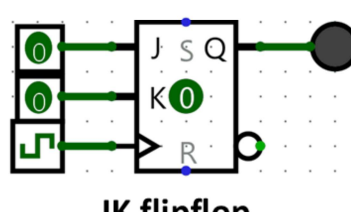
D flipflop



T flipflop



SR flipflop



JK flipflop

3

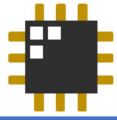
We also explored different types of flip-flops, each with unique behavior.

The **D flip-flop** (data/delay) has a single input, **D**, and a clock signal. Whatever value is on **D** at the rising edge of the clock is stored and appears at the **Q** output.

The **T flip-flop** (toggle) also has one input, **T**, which acts as a toggle enable. When **T = 0**, the output remains unchanged, but when **T = 1**, the output toggles to the complement of previous output.

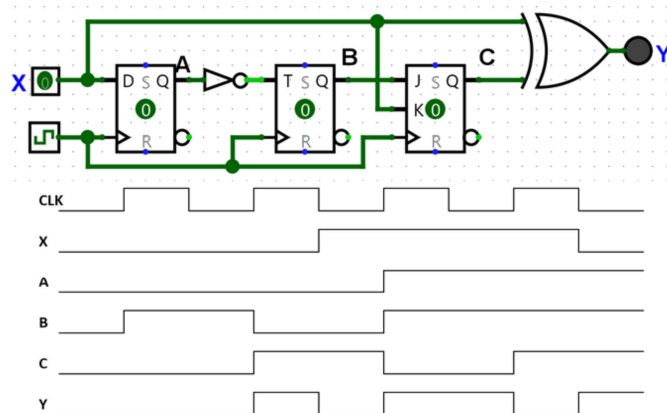
The **SR flip-flop** (set-reset) has two inputs, **S** (set) and **R** (reset). If **S = 1**, the output is set to 1; if **R = 1**, the output resets to 0; when both are 0, the output remains unchanged. However, **S = 1** and **R = 1** simultaneously is not allowed.

Lastly, the **JK flip-flop** is an improved version of the SR flip-flop, with inputs **J** (set) and **K** (reset). It functions like an SR flip-flop but resolves the invalid state by introducing a toggle feature: when both **J** and **K** are 1, the output flips between 0 and 1 on each clock cycle.



Last lecture

- Flipflop timing specifics



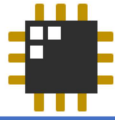
The input of the flipflop is checked *right before* ($t = 0 \text{ minus}$) the positive edge trigger.
The output of the flipflop is updated *right after* ($t = 0 \text{ plus}$) the positive edge trigger.

4

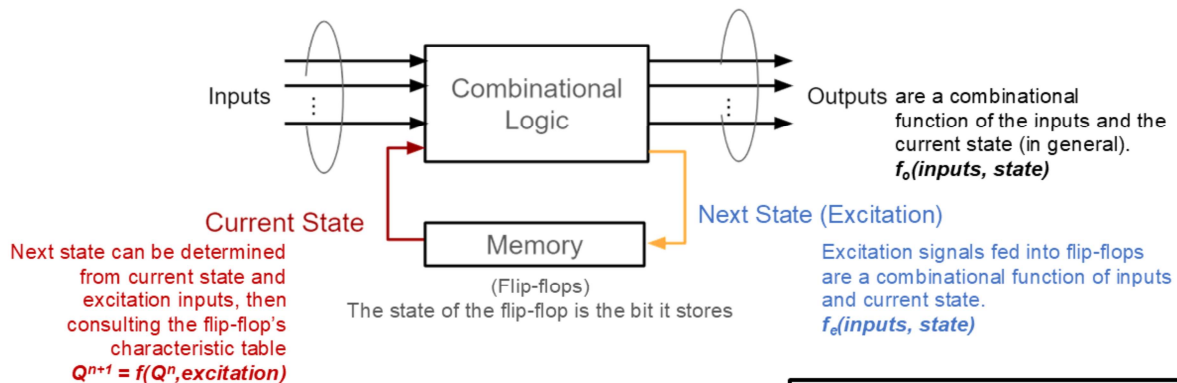
When working with flipflops, understanding timing is crucial to ensure reliable operation. Let's recall the following circuit with two flipflops chained together. The main thing to know is that the positive edge of the clock is just a single instant in time. Q copying D from the input A (the first flipflop) will not propagate all the way to C in a single positive edge. When B copies A on the positive edge, C copies B at the same time, but not at the same value B turned into on that same positive edge. The change of output happens at the same time. The flipflops both check the input values right before the positive edge and update right after the positive edge. Always!

You can also imagine the following description:

The input of the flipflop is checked *right before* ($t = 0 \text{ minus}$) the positive edge trigger.
The output of the flipflop is updated *right after* ($t = 0 \text{ plus}$) the positive edge trigger.
This is always how flipflops work whether in an ideal or non-ideal scenario, so hopefully you remember this.



Recall: Sequential circuit model

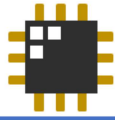


Note: flipflop **output** is also typically referred to as the flipflop **state**. We use these terms interchangeably.

Remember!
Flipflop inputs: **Excitation**
Flipflop outputs: **Current state**

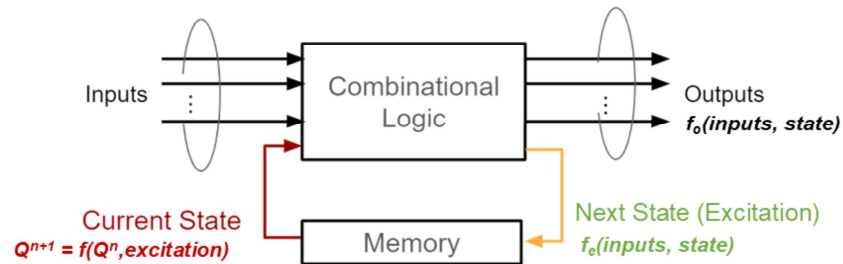
Now, let's recall the sequential circuit model. The sequential circuit model is different from the combinational circuit because it has memory, meaning its output depends not just on the current input but also on its previous state. This is achieved using flipflops, which stores the circuit's current state.

In this model, the input is fed an excitation signal, which is a combination function of inputs and current state and determines how the circuit should transition to the next state. The output, on the other hand, represents the current state of the system. The clock signal controls when the state updates, ensuring synchronization across the circuit.



Analyzing Synchronous Sequential Circuits

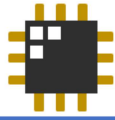
Given a circuit, we want to know what it does → state diagram*



Note: A **state diagram** is a graphical illustration of what the sequence of states would be following a sequence of input patterns. We'll show this later.

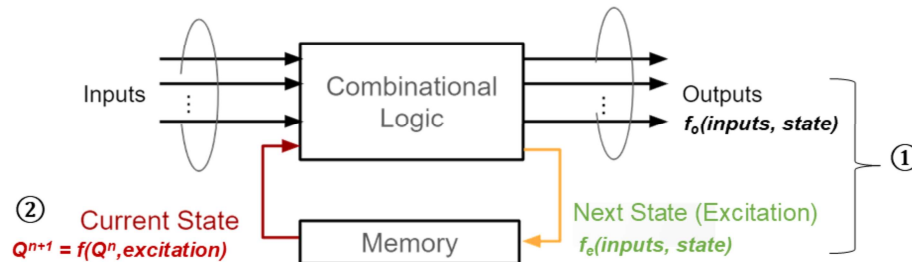
6

Since we know that the input (excitation) determines the next state and the output represents the current state, a state diagram is a great way to visualize how these transitions happen over time.



Analyzing Synchronous Sequential Circuits

Given a circuit, we want to know what it does → state diagram*



Using the model for analysis, we can obtain all the information needed by

- ① inspecting the combinational logic block and
- ② knowing how the memory elements behave (characteristic tables) to obtain
- ③ a state diagram

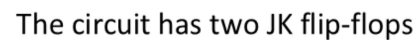
7

To fully understand how a sequential circuit behaves, we can analyze it step by step using a structured approach. This involves three components:

1. Inspecting the combinational logic block – this block consists of logic gates that process inputs to produce excitation signals for the memory elements. By examining this logic, we can determine the conditions that cause state transitions.
2. Knowing how the memory elements behave (characteristic tables) – since sequential circuits rely on flipflops to store state, we need to consider how these memory elements behave. Each type of flipflop follows a specific characteristic table, which defines how its output changes based on its input and clock signal. By using these tables, we can predict how the circuit transitions from one state to another.
3. Obtain a state diagram – once we understand the logic and memory behavior, we can map a state diagram, which visually represents how the circuit moves between states based on its inputs.



- Excitation equations $f_e(inputs, Q^n)$
- Output equations $f_o(inputs, Q^n)$



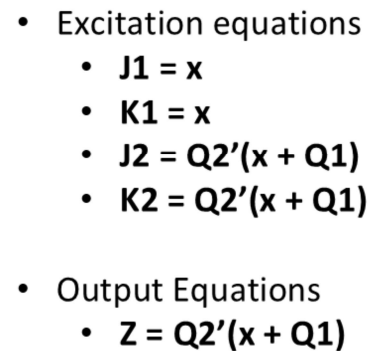
- Each flip-flop can store 1 bit, hence 1 state variable
- Therefore, this system can store 2 state variables Q2 Q1
- The system has up to 4 possible states, i.e. 00, 01, 10, 11

We start with the first step: inspecting the combinational logic block to derive the excitation and output equations.

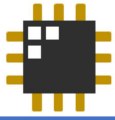
The given circuit consists of an input X, an output Z, and OR gate, an AND gate, and two JK flipflops. Since each JK flipflop stores one bit, the system has two state variables in total. This means the circuit can represent four possible states: 00, 01, 10, and 11.



- Excitation equations $f_e(inputs, Q^n)$
- Output equations $f_o(inputs, Q^n)$



These equations allow us to determine how the system transitions between states and produces its output.



Analysis Step 2: Create the transition table

Create a **transition table** for the system

- i. List all possible combinations of input values and present states Q^n
- ii. Using the excitation equations, derive the excitation values fed into flip-flops
- iii. Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- iv. Using the output equations, derive the output values

Main goal

You want to know what the **next flipflop outputs (next states) would be** for every possible combination of inputs and current flipflop outputs (current states)

10

Now, let's move on to the next step: creating the transition table.

Our goal here is to determine the **next state** of the flip-flops for every possible combination of **inputs** and **current states**. To achieve this, we follow a structured approach:

- i. List all possible combinations of input values and present states Q^n
- ii. Using the excitation equations, derive the excitation values fed into flip-flops
- iii. Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- iv. Using the output equations, derive the output values

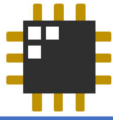
The main reason we're doing this is because we want to know what our next flipflop outputs would be given a certain input combination and current flipflop outputs. This would allow us to create the state diagram to visually describe what is happening.

To solidify our understanding, let's go through the example we had for step 1.



- i. List all possible combinations of input values and present states Q^n
- ii. Using the excitation equations, derive the excitation values fed into flip-flops
- iii. Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- iv. Using the output equations, derive the output values

11



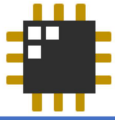
Analysis Step 2: Create the transition table

Input	Current State		Flip-flop Inputs				Next State		Output
	X	Q2	Q1	J2	K2	J1	K1	Q2	
0	0	0							
0	0	1							
0	1	0							
0	1	1							
1	0	0							
1	0	1							
1	1	0							
1	1	1							

- i. List all possible combinations of input values and present states Q^n

12

First, let's list all possible combinations of input (**x**) and current states (**Q1** and **Q2**). Since we are working with binary states, we can systematically count from **0 to 7 in binary** to cover all possible combinations.



Analysis Step 2: Create the transition table

Input X	Current State		Flip-flop Inputs				Next State		Output Z
	Q2	Q1	J2	K2	J1	K1	Q2	Q1	
0	0	0	0	0	0	0			
0	0	1	1	1	0	0			
0	1	0	0	0	0	0			
0	1	1	0	0	0	0			
1	0	0	1	1	1	1			
1	0	1	1	1	1	1			
1	1	0	0	0	1	1			
1	1	1	0	0	1	1			

$$J2 = K2 = Q2'(x + Q1) \quad J1 = K1 = x$$

- List all possible combinations of input values and present states Q^n
- Using the excitation equations, derive the excitation values fed into flip-flops

- Excitation equations
 - $J1 = x$
 - $K1 = x$
 - $J2 = Q2'(x + Q1)$
 - $K2 = Q2'(x + Q1)$

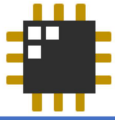
Now, let's use the **excitation equations** we derived earlier to determine the input values fed into the flip-flops.

Recall:

- $J1 = X, K1 = X$
- $J2 = Q2'(X + Q1), K2 = Q2'(X + Q1)$

For **J1 and K1**, since they are directly equal to **X**, we simply copy the values of **X** into the table.

For **J2 and K2**, we first take the complement of **Q2** (the second column) and **AND** it with the result of **X OR Q1**. This results in an input of **1** when **Q2 is 0 and X is 1** or when **Q2 is 0 and Q1 is 1**, and an input of **0** otherwise.



Analysis Step 2: Create the transition table

Input	Current State		Flip-flop Inputs				Next State		Output
X	Q2	Q1	J2	K2	J1	K1	Q2	Q1	Z
0	0	0	0	0	0	0	0		
0	0	1	1	1	0	0	1		
0	1	0	0	0	0	0	1		
0	1	1	0	0	0	0	1		
1	0	0	1	1	1	1	1		
1	0	1	1	1	1	1	1		
1	1	0	0	0	1	1	1		
1	1	1	0	0	1	1	1		

J	K	Q_{n+1}	What happened
0	0	Q_n	No change
0	1	0	Reset (0)
1	0	1	Set (1)
1	1	$(Q_n)'$	Toggle

- iii. Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- iv. Using the output equations, derive the output values

14

Now, let's move on to the next step: **determining the next state of each flip-flop** using the characteristic tables, the excitation values, and the current state.

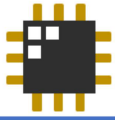
Since this example only involves **JK flip-flops**, we can refer to the characteristic table shown in the top right. Notably, the **J and K inputs for each flip-flop are identical**, meaning the flip-flop essentially functions like a **T flip-flop**. This simplifies our analysis:

- If the **J/K input is 0**, the flip-flop retains its **current state**.
- If the **J/K input is 1**, the flip-flop toggles to the **complement of its current state**.

Applying this rule, we can now determine the next state values for **Q2**. To do this, we simply check the value of **J2/K2**:

- If **J2/K2 is 0**, the next state of **Q2** remains the same as its current state.
- If **J2/K2 is 1**, the next state of **Q2** becomes the complement of its current state.

Using this approach, we derive the following values for the next state of **Q2**.



Analysis Step 2: Create the transition table

Input X	Current State		Flip-flop Inputs				Next State		Output Z
	Q2	Q1	J2	K2	J1	K1	Q2	Q1	
0	0	0	0	0	0	0	0	0	
0	0	1	1	1	0	0	1	1	
0	1	0	0	0	0	0	1	0	
0	1	1	0	0	0	0	1	1	
1	0	0	1	1	1	1	1	1	
1	0	1	1	1	1	1	1	0	
1	1	0	0	0	1	1	1	1	
1	1	1	0	0	1	1	1	0	

J	K	Q_{n+1}	What happened
0	0	Q_n	No change
0	1	0	Reset (0)
1	0	1	Set (1)
1	1	$(Q_n)'$	Toggle

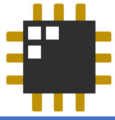
- iii. Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- iv. Using the output equations, derive the output values

15

Now, applying the rule for Q1:

- If **J1/K1 is 0**, the next state of **Q1** remains the same as its current state.
- If **J1/K1 is 1**, the next state of **Q1** becomes the complement of its current state.

Using this approach, we derive the following values for the next state of **Q1**.



Analysis Step 2: Create the transition table

Input X	Current State		Flip-flop Inputs				Next State		Output Z
	Q2	Q1	J2	K2	J1	K1	Q2	Q1	
0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	1	1	1
0	1	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	1	0
1	0	0	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	0	1
1	1	0	0	0	1	1	1	1	0
1	1	1	0	0	1	1	1	0	0

$$Z = Q2'(x + Q1)$$

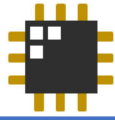
- List all possible combinations of input values and present states Q^n
- Using the excitation equations, derive the excitation values fed into flip-flops
- Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- Using the output equations, derive the output values**

- Output Equations
 - $Z = Q2'(x + Q1)$

Now, for the final part of **Step 2**, we use the **output equation** to determine the output values.

Recall that the output equation for this example is $Z = Q2'(x + Q1)$.

Here, we use the **current state values** of **Q2** and **Q1** to compute **Z**, which gives us the derived values in the table.



Analysis Step 2: Create the transition table

Input	Current State		Flip-flop Inputs				Next State		Output
X	Q2	Q1	J2	K2	J1	K1	Q2	Q1	Z
0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	1	1	1
0	1	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	1	0
1	0	0	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	0	1
1	1	0	0	0	1	1	1	1	0
1	1	1	0	0	1	1	1	0	0

What does this table say?

Assuming both flipflops have outputs = 0 (**current state 00**),

- if the input is **0**, the next flipflop outputs (next state) would be **00**
- if the input is **1**, the next flipflop outputs (next state) would be **11**

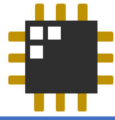
Note that going to the next state requires the trigger event (e.g. positive edge of the clock) since the flipflops must update their outputs

17

Now, let's analyze the table, assuming both flip-flops start with outputs **0** (current state **00**).

Observing the transitions, we see that:

- When the **input is 0**, the next flip-flop outputs remain **00** (highlighted in blue).
- When the **input is 1**, the next flip-flop outputs change to **11** (highlighted in yellow).



Analysis Step 2: Create the transition table

Input	Current State		Flip-flop Inputs				Next State		Output
X	Q2	Q1	J2	K2	J1	K1	Q2	Q1	Z
0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	1	1	1
0	1	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	1	0
1	0	0	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	0	1
1	1	0	0	0	1	1	1	1	0
1	1	1	0	0	1	1	1	0	0

What does this table say?

Assuming both flipflops have outputs = 1 (**current state 11**),

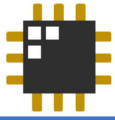
- if the input is **0**, the next flipflop outputs (next state) would be **11**
- if the input is **1**, the next flipflop outputs (next state) would be **10**

Note that going to the next state requires the trigger event (e.g. positive edge of the clock) since the flipflops must update their outputs

18

Now, let's examine the case where the current state of the flipflops is **11**.

- When the **input is 0**, the next state remains **11**.
- When the **input is 1**, the next state transitions to **10**.



Analysis Step 2: Create the transition table

Input	Current State		Flip-flop Inputs				Next State		Output
X	Q2	Q1	J2	K2	J1	K1	Q2	Q1	Z
0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	1	1	1
0	1	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	1	0
1	0	0	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	0	1
1	1	0	0	0	1	1	1	1	0
1	1	1	0	0	1	1	1	0	0

What does this table say?

This tells us what the next flipflop outputs (next state) would be given the current flipflop outputs (current state) and the current input, ***for all possible permutations.***

Instead of a table, a drawing would be nice 😊

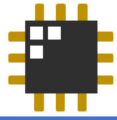
19

So, what does this table represent?

It provides a clear mapping of how the flip-flop outputs will transition based on the current state and input, covering all possible combinations.

This table essentially defines the **behavior of the sequential circuit**, showing how the **current state and input** determine the **next state**. By analyzing these transitions, we can better understand how the system evolves over time.

While this table perfectly describes the behavior of the circuit, a better illustration can be created to visualize what exactly is happening. This leads us to the creation of a state diagram.

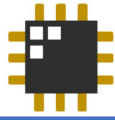


Analysis Step 3: Draw the state diagram

A state transition diagram represents each state as a bubble and transitions (where to go next) as arrows.

20

While the table is useful, a **state diagram** would provide a more intuitive way to visualize these transitions. A diagram allows us to **see the flow of states** at a glance, making it easier to follow the logic behind the circuit's operation.



Analysis Step 3: Draw the state diagram

- Number of states in the diagram = number of unique values or entries in the current state column (or next state column) of the state transition table

Input X	Current State		Flip-flop Inputs				Next State		Output Z
	Q2	Q1	J2	K2	J1	K1	Q2	Q1	
0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	1	1	1
0	1	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	1	0
1	0	0	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	0	1
1	1	0	0	0	1	1	1	1	0
1	1	1	0	0	1	1	1	0	0

We have 4 unique states: 00, 01, 10, 11

We can assign codenames to them

State assignment:

- Let **S0** represent **Q2 = 0 & Q1 = 0**
- Let **S1** represent **Q2 = 0 & Q1 = 1**
- Let **S2** represent **Q2 = 1 & Q1 = 0**
- Let **S3** represent **Q2 = 1 & Q1 = 1**

21

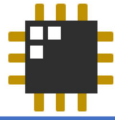
Before drawing a state transition diagram, we first assign a **codename** to each state, a process known as **state assignment**. This ensures that every state in the system is uniquely identified using state variables.

The total number of states depends on the number of flip-flops, with **n** flip-flops allowing for up to **2ⁿ** possible states. These assigned codenames replace the state labels in the **state transition table** for easier reference.

In our example, we use the following assignments:

- S0** for **Q2 = 0, Q1 = 0 (00)**
- S1** for **Q2 = 0, Q1 = 1 (01)**
- S2** for **Q2 = 1, Q1 = 0 (10)**
- S3** for **Q2 = 1, Q1 = 1 (11)**

Notice how we simply use the **decimal of the binary representation** of the flip-flop outputs? This makes it easier to reference and analyze transitions in the system.



Analysis Step 3: Draw the state diagram

- Number of states in the diagram = number of unique values or entries in the current state column (or next state column) of the state transition table

Input X	Current State		Flip-flop Inputs				Next State		Output Z
	Q2	Q1	J2	K2	J1	K1	Q2	Q1	
0	S0		0	0	0	0	S0		0
0	S1		1	1	0	0	S3		1
0	S2		0	0	0	0	S2		0
0	S3		0	0	0	0	S3		0
1	S0		1	1	1	1	S3		1
1	S1		1	1	1	1	S2		1
1	S2		0	0	1	1	S3		0
1	S3		0	0	1	1	S2		0

We have 4 unique states: 00, 01, 10, 11

We can assign codenames to them

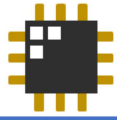
State assignment:

- Let **S0** represent **Q2 = 0 & Q1 = 0**
- Let **S1** represent **Q2 = 0 & Q1 = 1**
- Let **S2** represent **Q2 = 1 & Q1 = 0**
- Let **S3** represent **Q2 = 1 & Q1 = 1**

*At this point, the excitation column (flipflop inputs) is not relevant anymore and can be removed for simplicity

Now, we map the state assignments onto the **state transition table**, making it easier to translate into a **state transition diagram**.

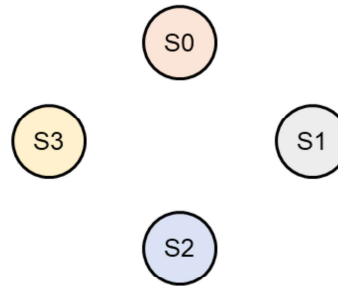
At this point, the **flip-flop inputs (excitation values)** are no longer needed, as our focus shifts to how the **current states transition** to the **next states** and how the **output is determined** based on the input conditions.



Analysis Step 3: Draw the state diagram

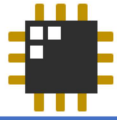
- Number of states in the diagram = number of unique values or entries in the current state column (or next state column) of the state transition table

Input	Current State		Next State		Output
X	Q2	Q1	Q2	Q1	Z
0	S0		S0		0
0	S1		S3		1
0	S2		S2		0
0	S3		S3		0
1	S0		S3		1
1	S1		S2		1
1	S2		S3		0
1	S3		S2		0



Now, we just create arrows from one current state pointing to the next states for every input combination!

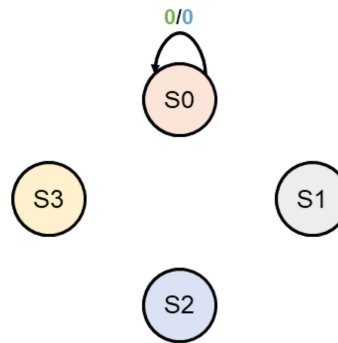
Now, we represent the states in the **state transition diagram**, where each state is depicted as a **circle**, and transitions are illustrated with **arrows** labeled with the corresponding **input/output** values.



Analysis Step 3: Draw the state diagram

- We would indicate **input/output** on the arrows on a row by row basis on the state transition table

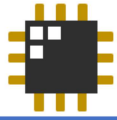
Input	Current State		Next State		Output
X	Q2	Q1	Q2	Q1	Z
0	S0		S0		0
0	S1		S3		1
0	S2		S2		0
0	S3		S3		0
1	S0		S3		1
1	S1		S2		1
1	S2		S3		0
1	S3		S2		0



24

Let's begin with **state S0**.

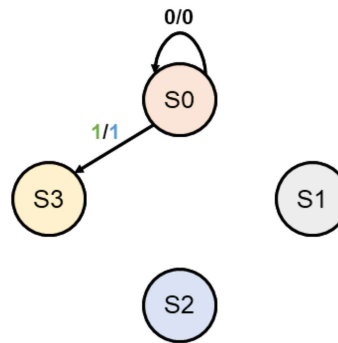
- When the **input is 0**, the state remains **S0**, so the arrow loops back to itself, labeled **0/0** to indicate that both the input and output are 0.
- When the **input is 1**, the state transitions to **S3**, represented by an arrow from **S0 to S3**, labeled **1/1** since the input is 1 and the output is also 1.



Analysis Step 3: Draw the state diagram

- We would indicate **input/output** on the arrows on a row by row basis on the state transition table

Input	Current State		Next State		Output
X	Q2	Q1	Q2	Q1	Z
0	S0		S0		0
0	S1		S3		1
0	S2		S2		0
0	S3		S3		0
1	S0		S3		1
1	S1		S2		1
1	S2		S3		0
1	S3		S2		0



Each state will have 2^n arrows going out of it, where n is the number of input bits

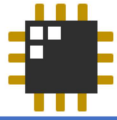
25

Let's begin with **state S0**.

- When the **input is 0**, the state remains **S0**, so the arrow loops back to itself, labeled **0/0** to indicate that both the input and output are 0.
- When the **input is 1**, the state transitions to **S3**, represented by an arrow from **S0 to S3**, labeled **1/1** since the input is 1 and the output is also 1.

It is helpful to note that since we have 1-bit input, there are two possible values, 0 or 1. Thus, each state will have two arrows going out of it, each corresponding to the input value.

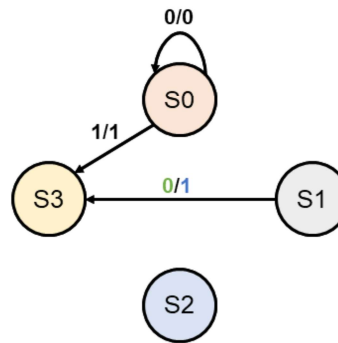
In general, states will have 2^n arrows going out of it, where n is the number of input bits.



Analysis Step 3: Draw the state diagram

- We would indicate **input/output** on the arrows on a row by row basis on the state transition table

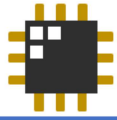
Input	Current State	Next State	Output
X	Q2	Q1	Z
0	S0	S0	0
0	S1	S3	1
0	S2	S2	0
0	S3	S3	0
1	S0	S3	1
1	S1	S2	1
1	S2	S3	0
1	S3	S2	0



26

Next, let's look at **state S1**.

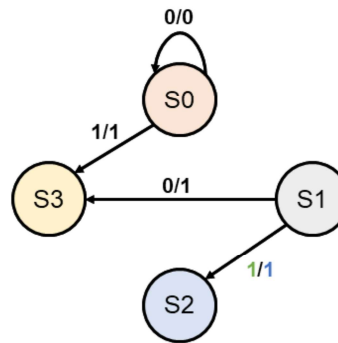
- When the **input is 0**, the state transitions to **S3**, and the output is **1**, so we draw an arrow from **S1 to S3**, labeled **0/1**.
- When the **input is 1**, the state moves to **S2**, with an output of **1**, so we add an arrow from **S1 to S2**, labeled **1/1**.



Analysis Step 3: Draw the state diagram

- We would indicate **input/output** on the arrows on a row by row basis on the state transition table

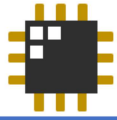
Input	Current State	Next State	Output
X	Q2	Q1	Z
0	S0	S0	0
0	S1	S3	1
0	S2	S2	0
0	S3	S3	0
1	S0	S3	1
1	S1	S2	1
1	S2	S3	0
1	S3	S2	0



27

Next, let's look at **state S1**.

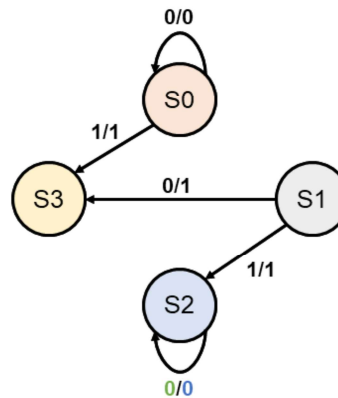
- When the **input is 0**, the state transitions to **S3**, and the output is **1**, so we draw an arrow from **S1 to S3**, labeled **0/1**.
- When the **input is 1**, the state moves to **S2**, with an output of **1**, so we add an arrow from **S1 to S2**, labeled **1/1**.



Analysis Step 3: Draw the state diagram

- We would indicate **input/output** on the arrows on a row by row basis on the state transition table

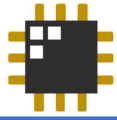
Input	Current State	Next State	Output
X	Q2	Q1	Z
0	S0	S0	0
0	S1	S3	1
0	S2	S2	0
0	S3	S3	0
1	S0	S3	1
1	S1	S2	1
1	S2	S3	0
1	S3	S2	0



28

Now, let's examine **state S2**.

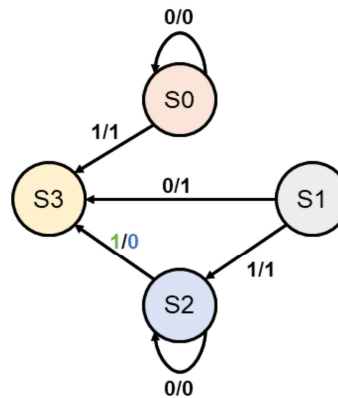
- When the **input is 0**, the state remains **S2**, and the output is **0**, so we draw a **looping arrow** back to **S2**, labeled **0/0**.
- When the **input is 1**, the state transitions to **S3**, with an output of **0**, so we add an **arrow from S2 to S3**, labeled **1/0**.



Analysis Step 3: Draw the state diagram

- We would indicate **input/output** on the arrows on a row by row basis on the state transition table

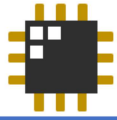
Input	Current State		Next State		Output
X	Q2	Q1	Q2	Q1	Z
0	S0		S0		0
0	S1		S3		1
0	S2		S2		0
0	S3		S3		0
1	S0		S3		1
1	S1		S2		1
1	S2		S3		0
1	S3		S2		0



29

Now, let's examine **state S2**.

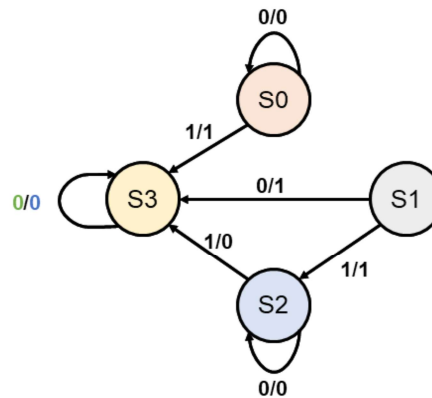
- When the **input is 0**, the state remains **S2**, and the output is **0**, so we draw a **looping arrow** back to **S2**, labeled **0/0**.
- When the **input is 1**, the state transitions to **S3**, with an output of **0**, so we add an **arrow from S2 to S3**, labeled **1/0**.



Analysis Step 3: Draw the state diagram

- We would indicate **input/output** on the arrows on a row by row basis on the state transition table

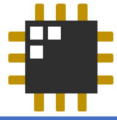
Input	Current State	Next State	Output
X	Q2	Q1	Z
0	S0	S0	0
0	S1	S3	1
0	S2	S2	0
0	S3	S3	0
1	S0	S3	1
1	S1	S2	1
1	S2	S3	0
1	S3	S2	0



30

Lastly, let's analyze **state S3**.

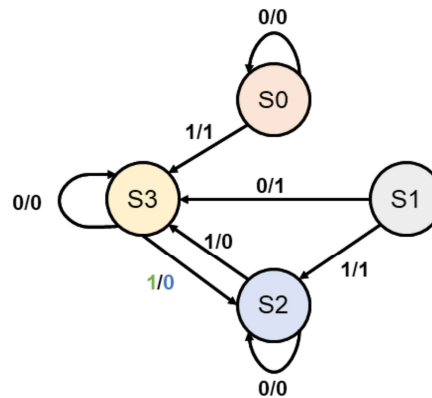
- When the **input is 0**, the state remains **S3**, and the output is **0**, so we draw a **looping arrow** back to **S3**, labeled **0/0**.
- When the **input is 1**, the state transitions to **S2**, with an output of **0**, so we draw an **arrow from S3 to S2**, labeled **1/0**.



Analysis Step 3: Draw the state diagram

- We would indicate **input/output** on the arrows on a row by row basis on the state transition table

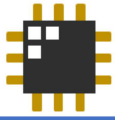
Input	Current State		Next State		Output
X	Q2	Q1	Q2	Q1	Z
0	S0		S0		0
0	S1		S3		1
0	S2		S2		0
0	S3		S3		0
1	S0		S3		1
1	S1		S2		1
1	S2		S3		0
1	S3		S2		0



31

Lastly, let's analyze **state S3**.

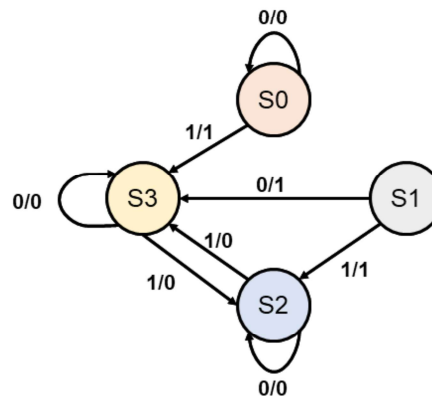
- When the **input is 0**, the state remains **S3**, and the output is **0**, so we draw a **looping arrow** back to **S3**, labeled **0/0**.
- When the **input is 1**, the state transitions to **S2**, with an output of **0**, so we draw an **arrow from S3 to S2**, labeled **1/0**.



Analysis Step 3: Draw the state diagram

The number of **arrows going out of the state** should be 2^n , where n is the number of input bits

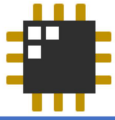
The **total number of arrows in the diagram** should be the **number of rows** in the state transition table



32

In a state transition diagram, the number of outgoing arrows from each state should be 2^n , where n is the number of input bits. This is because each input combination defines a unique transition for that state. For example, if there is only one input bit ($n=1$), each state will have **2 outgoing arrows**—one for input **0** and one for input **1**.

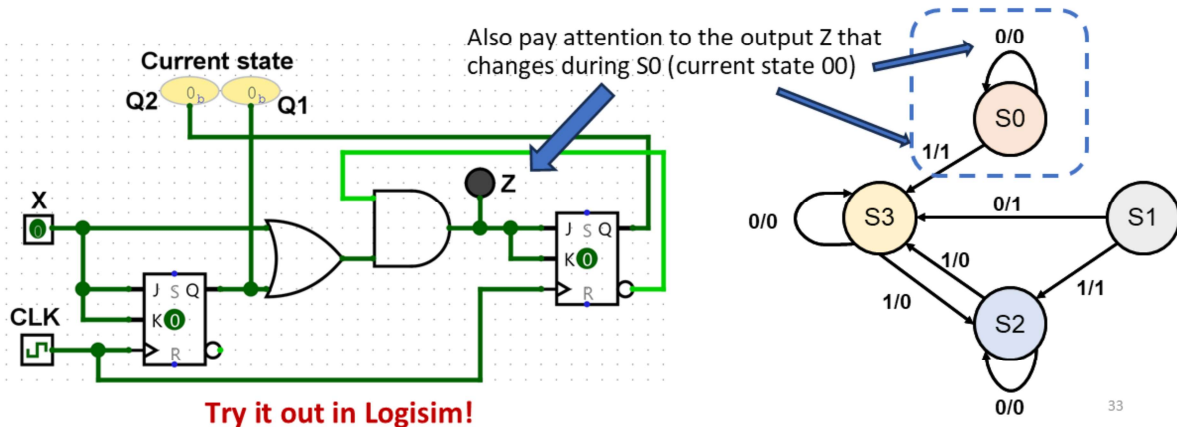
Additionally, the **total number of arrows** in the entire diagram should match the **number of rows in the state transition table**. Since the table lists every possible combination of current states and inputs, each row corresponds to a unique transition, ensuring that all state changes are accurately represented in the diagram.



Analysis Step 3: Draw the state diagram

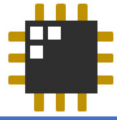
Note that you only go from one circle to another on every trigger event!

- Changing the input simply changes the arrow you're taking, but transferring circles needs the trigger (e.g. positive edge of the clock) to happen every time

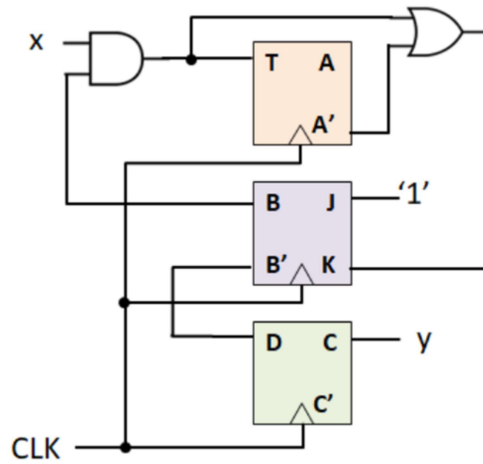


In a state transition diagram, movement from one state (circle) to another **only occurs on a trigger event**, such as the **positive edge of the clock** in synchronous sequential circuits. This means that even if the input changes, the system does not immediately switch states. Instead, the change in input **determines which transition (arrow) will be taken** once the trigger event occurs.

Think of it as choosing a path before stepping forward—changing the input **selects the arrow**, but the actual transition **only happens when the clock edge triggers it**. This ensures that state transitions occur in a controlled, predictable manner, preventing unintended or asynchronous changes in the system.



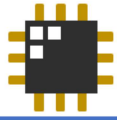
Another analysis example



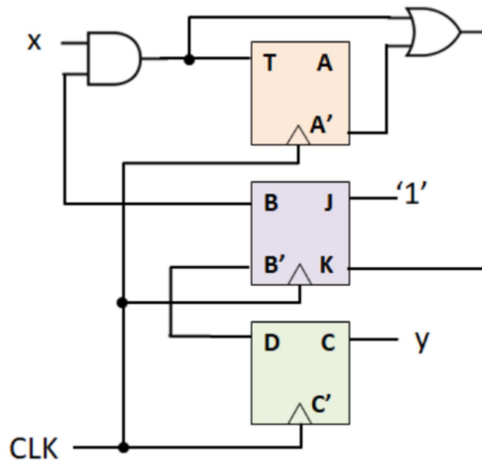
- 1) Study the circuit and obtain **equations**
- 2) Create a **transition table** for the system
- 3) Draw a **state diagram**

34

Now, let's go through another example, applying the same step-by-step process we previously discussed.



Analysis Step 1: Write equations



Study the circuit and obtain **equations**

- Excitation equations $f_e(inputs, Q^n)$
- Output equations $f_o(inputs, Q^n)$
- Excitation equations
 - $T = Bx$
 - $J = 1$
 - $K = A' + Bx$
 - $D = B'$
- Output equations
 - $y = C$

35

First, let's analyze the circuit to derive the excitation and output equations.

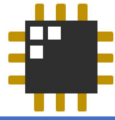
This circuit consists of an input x , an output y , an AND gate, an OR gate, a T flip-flop, a JK flip-flop, and a D flip-flop. Now, let's examine how these components interact to determine the necessary equations.

For the T flip-flop, its input comes from an AND gate with inputs B and x , giving us $T = Bx$.

For the JK flip-flop, the J input is always 1, while the K input comes from an OR gate with inputs A' and AND gate with inputs x and B , resulting in $J = 1$ and $K = A' + Bx$.

For the D flip-flop, its input is simply B' , so we have $D = B'$.

Lastly, the output equation is $y = C$.



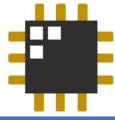
Analysis Step 2: Create the transition table

Input	Current State			Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y

- List all possible combinations of input values and present states Q^n
- Using the excitation equations, derive the excitation values fed into flip-flops
- Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- Using the output equations, derive the output values

36

Now, let's begin constructing the transition table by applying the step-by-step method we previously discussed.



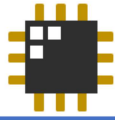
Analysis Step 2: Create the transition table

Input	Current State			Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0								
0	0	0	1								
0	0	1	0								
0	0	1	1								
0	1	0	0								
0	1	0	1								
0	1	1	0								
0	1	1	1								
1	0	0	0								
1	0	0	1								
1	0	1	0								
1	0	1	1								
1	1	0	0								
1	1	0	1								
1	1	1	0								
1	1	1	1								

- i. List all possible combinations of input values and present states Q^n

37

Again, the first step is to list down all possible combinations of input value (x) and current states (A, B, and C). Note that we are working with binary states, we can systematically count from **0 to 15 in binary** to cover all possible combinations.



Analysis Step 2: Create the transition table

Input		Current State			Flip-flop Inputs				Next State			Output
X		A	B	C	T	J	K	D	A	B	C	Y
0		0	0	0								
0		0	0	1								
0		0	1	0								
0		0	1	1								
0		1	0	0								
0		1	0	1								
0		1	1	0								
0		1	1	1								
1		0	0	0								
1		0	0	1								
1		0	1	0								
1		0	1	1								
1		1	0	0								
1		1	0	1								
1		1	1	0								
1		1	1	1								

i. List all possible combinations of input values and present states Q^n

ii. **Using the excitation equations, derive the excitation values fed into flip-flops**

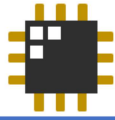
Excitation equations

- $T = Bx$
- $J = 1$
- $K = A' + Bx$
- $D = B'$

38

Using the excitation equations we derived earlier, we can now determine the input values fed into the flip-flops.

- $T = Bx \rightarrow$ The input is 1 when both $B = 1$ and $x = 1$; otherwise, it is 0.
- $J = 1 \rightarrow$ The input is always 1.
- $K = A' + Bx \rightarrow$ The input is 1 if $A = 0$ or if $B = 1$ and $x = 1$; otherwise, it is 0.
- $D = B' \rightarrow$ The input is 1 when $B = 0$.



Analysis Step 2: Create the transition table

Input		Current State			Flip-flop Inputs				Next State			Output
X		A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	0	1	1	1				
0	0	0	0	1	0	1	1	1				
0	0	0	1	0	0	1	1	0				
0	0	0	1	1	0	1	1	0				
0	1	0	0	0	0	1	0	1				
0	1	0	0	1	0	1	0	1				
0	1	1	0	0	0	1	0	0				
0	1	1	1	0	0	1	0	0				
1	0	0	0	0	0	1	1	1				
1	0	0	0	1	0	1	1	1				
1	0	1	0	0	1	1	1	0				
1	0	1	0	1	1	1	1	0				
1	1	0	0	0	0	1	0	1				
1	1	0	0	1	0	1	0	1				
1	1	1	0	0	1	1	1	0				
1	1	1	0	1	1	1	1	0				
1	1	1	1	1	1	1	1	0				

i. List all possible combinations of input values and present states Q^n

ii. Using the excitation equations, derive the excitation values fed into flip-flops

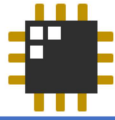
Excitation equations

- $T = Bx$
- $J = 1$
- $K = A' + Bx$
- $D = B'$

39

Using the excitation equations we derived earlier, we can now determine the input values fed into the flip-flops.

- $T = Bx \rightarrow$ The input is 1 when both $B = 1$ and $x = 1$; otherwise, it is 0.
- $J = 1 \rightarrow$ The input is always 1.
- $K = A' + Bx \rightarrow$ The input is 1 if $A = 0$ or if $B = 1$ and $x = 1$; otherwise, it is 0.
- $D = B' \rightarrow$ The input is 1 when $B = 0$.



Analysis Step 2: Create the transition table

Input		Current State			Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y	
0	0	0	0	0	1	1	1					
0	0	0	1	0	1	1	1					
0	0	1	0	0	1	1	0					
0	0	1	1	0	1	1	0					
0	1	0	0	0	1	0	1					
0	1	0	1	0	1	0	1					
0	1	1	0	0	1	0	0					
0	1	1	1	0	1	0	0					
1	0	0	0	0	1	1	1					
1	0	0	1	0	1	1	1					
1	0	1	0	1	1	1	0					
1	0	1	1	1	1	1	0					
1	1	0	0	0	1	0	1					
1	1	0	1	0	1	0	1					
1	1	1	0	1	1	1	0					
1	1	1	1	1	1	1	0					

T	Q_{n+1}	What happened
0	Q_n	No change
1	$(Q_n)'$	Toggle

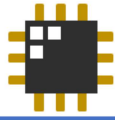
- iii. Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- iv. Using the output equations, derive the output values

40

Now, let's proceed with determining the next state of each flip-flop using the characteristic tables, excitation values, and the current state.

Starting with the T flip-flop, we refer to its characteristic table. The next state of **A** depends on the value of **T**:

- If **T = 0**, the next state of **A** remains the same as its current state.
- If **T = 1**, the next state of **A** becomes the complement of its current state.



Analysis Step 2: Create the transition table

Input		Current State		Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	1	1	1	0			
0	0	0	1	0	1	1	1	0			
0	0	1	0	0	1	1	0	0			
0	0	1	1	0	1	1	0	0			
0	1	0	0	0	1	0	1	1			
0	1	0	1	0	1	0	1	1			
0	1	1	0	0	1	0	0	1			
0	1	1	1	0	1	0	0	1			
1	0	0	0	0	1	1	1	0			
1	0	0	1	0	1	1	1	0			
1	0	1	0	1	1	1	0	1			
1	0	1	1	1	1	1	0	1			
1	1	0	0	0	1	0	1	1			
1	1	0	1	0	1	0	1	1			
1	1	1	0	1	1	1	0	0			
1	1	1	1	1	1	1	0	0			

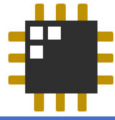
T	Q_{n+1}	What happened
0	Q_n	No change
1	$(Q_n)'$	Toggle

- iii. Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- iv. Using the output equations, derive the output values

41

Again, referring to the characteristic table, we derive the following values for the next state of **A**.

- If **T** is **0**, the next state of **A** remains the same as its current state.
- If **T** is **1**, the next state of **A** becomes the complement of its current state.



Analysis Step 2: Create the transition table

Input		Current State		Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	1	1	1	0			
0	0	0	1	0	1	1	1	0			
0	0	1	0	0	1	1	0	0			
0	0	1	1	0	1	1	0	0			
0	1	0	0	0	1	0	1	1			
0	1	0	1	0	1	0	1	1			
0	1	1	0	0	1	0	0	1			
0	1	1	1	0	1	0	0	1			
1	0	0	0	0	1	1	1	0			
1	0	0	1	0	1	1	1	0			
1	0	1	0	1	1	1	0	1			
1	0	1	1	1	1	1	0	1			
1	1	0	0	0	1	0	1	1			
1	1	0	1	0	1	0	1	1			
1	1	1	0	1	1	1	0	0			
1	1	1	1	1	1	1	0	0			

J	K	Q_{n+1}	What happened
0	0	Q_n	No change
0	1	0	Reset (0)
1	0	1	Set (1)
1	1	$(Q_n)'$	Toggle

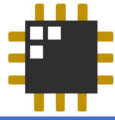
- iii. Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- iv. Using the output equations, derive the output values

42

Now, let's analyze the JK flip-flop using its characteristic table.

The next state of **B** depends on the **J** and **K** inputs. In this example, there are only two possible cases for **J** and **K**: **11** and **10**.

- When **J = 1** and **K = 1**, the next state of **B** becomes the complement of its current state.
- When **J = 1** and **K = 0**, the next state of **B** is simply **1**.



Analysis Step 2: Create the transition table

Input		Current State		Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	1	1	1	0	1		
0	0	0	1	0	1	1	1	0	1		
0	0	1	0	0	1	1	0	0	0		
0	0	1	1	0	1	1	0	0	0		
0	1	0	0	0	1	0	1	1	1		
0	1	0	1	0	1	0	1	1	1		
0	1	1	0	0	1	0	0	1	1		
0	1	1	1	0	1	0	0	1	1		
1	0	0	0	0	1	1	1	0	1		
1	0	0	1	0	1	1	1	0	1		
1	0	1	0	1	1	1	0	1	0		
1	0	1	1	1	1	1	0	1	0		
1	1	0	0	0	1	0	1	1	1		
1	1	0	1	0	1	0	1	1	1		
1	1	1	0	1	1	1	0	0	0		
1	1	1	1	1	1	1	0	0	0		

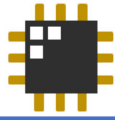
J	K	Q_{n+1}	What happened
0	0	Q_n	No change
0	1	0	Reset (0)
1	0	1	Set (1)
1	1	$(Q_n)'$	Toggle

- iii. Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- iv. Using the output equations, derive the output values

43

Using the characteristic table, we can now determine the next state values for **B**, resulting in the following values.

- When **J = 1** and **K = 1**, the next state of **B** becomes the complement of its current state.
- When **J = 1** and **K = 0**, the next state of **B** is simply **1**.



Analysis Step 2: Create the transition table

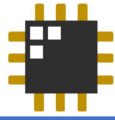
Input	Current State			Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	1	1	1	0	1		
0	0	0	1	0	1	1	1	0	1		
0	0	1	0	0	1	1	0	0	0		
0	0	1	1	0	1	1	0	0	0		
0	1	0	0	0	1	0	1	1	1		
0	1	0	1	0	1	0	1	1	1		
0	1	1	0	0	1	0	0	1	1		
0	1	1	1	0	1	0	0	1	1		
1	0	0	0	0	1	1	1	0	1		
1	0	0	1	0	1	1	1	0	1		
1	0	1	0	1	1	1	0	1	0		
1	0	1	1	1	1	1	0	1	0		
1	1	0	0	0	1	0	1	1	1		
1	1	0	1	0	1	0	1	1	1		
1	1	1	0	1	1	1	0	0	0		
1	1	1	1	1	1	1	0	0	0		

D	Q_{n+1}	What happened
0	0	Copy D (0)
1	1	Copy D (1)

- iii. Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- iv. Using the output equations, derive the output values

44

Finally, let's determine the next state for the **D** flip-flop. Since a D flip-flop directly follows its input, the next state will simply be the value of **D**.



Analysis Step 2: Create the transition table

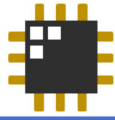
Input	Current State			Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	1	1	1	0	1	1	
0	0	0	1	0	1	1	1	0	1	1	
0	0	1	0	0	1	1	0	0	0	0	
0	0	1	1	0	1	1	0	0	0	0	
0	1	0	0	0	1	0	1	1	1	1	
0	1	0	1	0	1	0	1	1	1	1	
0	1	1	0	0	1	0	0	1	1	0	
0	1	1	1	0	1	0	0	1	1	0	
1	0	0	0	0	1	1	1	0	1	1	
1	0	0	1	0	1	1	1	0	1	1	
1	0	1	0	1	1	1	0	1	0	0	
1	0	1	1	1	1	1	0	1	0	0	
1	1	0	0	0	1	0	1	1	1	1	
1	1	0	1	0	1	0	1	1	1	1	
1	1	1	0	1	1	1	0	0	0	0	
1	1	1	1	1	1	1	0	0	0	0	

D	Q_{n+1}	What happened
0	0	Copy D (0)
1	1	Copy D (1)

- iii. Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- iv. Using the output equations, derive the output values

45

Again, we just copied what D is for the next state of C.



Analysis Step 2: Create the transition table

Input	Current State			Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	1	1	1	0	1	1	
0	0	0	1	0	1	1	1	0	1	1	
0	0	1	0	0	1	1	0	0	0	0	
0	0	1	1	0	1	1	0	0	0	0	
0	1	0	0	0	1	0	1	1	1	1	
0	1	0	1	0	1	0	1	1	1	1	
0	1	1	0	0	1	0	0	1	1	0	
0	1	1	1	0	1	0	0	1	1	0	
1	0	0	0	0	1	1	1	0	1	1	
1	0	0	1	0	1	1	1	0	1	1	
1	0	1	0	1	1	1	0	1	0	0	
1	0	1	1	1	1	1	0	1	0	0	
1	1	0	0	0	1	0	1	1	1	1	
1	1	0	1	0	1	0	1	1	1	1	
1	1	1	0	1	1	1	0	0	0	0	
1	1	1	1	1	1	1	0	0	0	0	

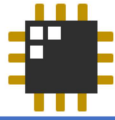
- List all possible combinations of input values and present states Q^n
- Using the excitation equations, derive the excitation values fed into flip-flops
- Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- Using the output equations, derive the output values**

• Output equations

• $Y = C$

46

Now, we will determine the output values using the output equation. Since we derived $Y = C$ earlier, the output Y will simply match the current state of C .



Analysis Step 2: Create the transition table

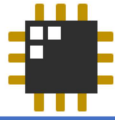
Input	Current State			Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	1	1	1	0	1	1	0
0	0	0	1	0	1	1	1	0	1	1	1
0	0	1	0	0	1	1	0	0	0	0	0
0	0	1	1	0	1	1	0	0	0	0	1
0	1	0	0	0	1	0	1	1	1	1	0
0	1	0	1	0	1	0	1	1	1	1	1
0	1	1	0	0	1	0	0	1	1	0	0
0	1	1	1	0	1	0	0	1	1	0	1
1	0	0	0	0	1	1	1	0	1	1	0
1	0	0	1	0	1	1	1	0	1	1	1
1	0	1	0	1	1	1	0	1	0	0	0
1	0	1	1	1	1	1	0	1	0	0	1
1	1	0	0	0	1	0	1	1	1	1	0
1	1	0	1	0	1	0	1	1	1	1	1
1	1	1	0	1	1	1	0	0	0	0	0
1	1	1	1	1	1	1	0	0	0	0	1

- List all possible combinations of input values and present states Q^n
- Using the excitation equations, derive the excitation values fed into flip-flops
- Using the characteristic tables, and given the excitation values and current state, determine the next state Q^{n+1} of each flip-flop
- Using the output equations, derive the output values**

• Output equations
• $Y = C$

47

Now, we determine the output values using the output equation. Since we derived $Y = C$ earlier, the output Y will simply match the current state of C .



Analysis Step 3: Draw the state diagram

Input	Current State			Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	1	1	1	0	1	1	0
0	0	0	1	0	1	1	1	0	1	1	1
0	0	1	0	0	1	1	0	0	0	0	0
0	0	1	1	0	1	1	0	0	0	0	1
0	1	0	0	0	1	0	1	1	1	1	0
0	1	0	1	0	1	0	1	1	1	1	1
0	1	1	0	0	1	0	0	1	1	0	0
0	1	1	1	0	1	0	0	1	1	0	1
1	0	0	0	0	1	1	1	0	1	1	0
1	0	0	1	0	1	1	1	0	1	1	1
1	0	1	0	1	1	1	0	1	0	0	0
1	0	1	1	1	1	1	0	1	0	0	1
1	1	0	0	0	1	0	1	1	1	1	0
1	1	0	1	0	1	0	1	1	1	1	1
1	1	1	0	1	1	1	0	0	0	0	0
1	1	1	1	1	1	1	0	0	0	0	1

You can already draw the state diagram straight from this table.

State assignment (putting codenames to the unique binary states) is *optional*.

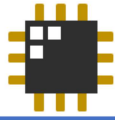
Remember:

- There should be **8 states in total** (corresponding to flipflop outputs)
- There should be **16 arrows** (since each state has 2 arrows, corresponding to the 1-bit input, coming out of each state)

48

At this point, you can directly draw the state transition diagram based on the transition table without needing to assign codenames to the states. State assignment, which involves giving each unique binary state a label (e.g., S0, S1, etc.), is optional and mainly helps with readability.

In this example, since we have three flip-flops, there are **8 possible states** ($2^3 = 8$), representing all combinations of flip-flop outputs. Additionally, since each state has **two possible transitions** based on the 1-bit input (0 or 1), the diagram should have **16 arrows** in total, ensuring every possible state transition is accounted for.



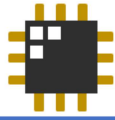
Analysis Step 3: Draw the state diagram

Input	Current State			Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	1	1	1	0	1	1	0
0	0	0	1	0	1	1	1	0	1	1	1
0	0	1	0	0	1	1	0	0	0	0	0
0	0	1	1	0	1	1	0	0	0	0	1
0	1	0	0	0	1	0	1	1	1	1	0
0	1	0	1	0	1	0	1	1	1	1	1
0	1	1	0	0	1	0	0	1	1	0	0
0	1	1	1	0	1	0	0	1	1	0	1
1	0	0	0	0	1	1	1	0	1	1	0
1	0	0	1	0	1	1	1	0	1	1	1
1	0	1	0	1	1	1	0	1	0	0	0
1	0	1	1	1	1	1	0	1	0	0	1
1	1	0	0	0	1	0	1	1	1	1	0
1	1	0	1	0	1	0	1	1	1	1	1
1	1	1	0	1	1	1	0	0	0	0	0
1	1	1	1	1	1	1	0	0	0	0	1

Try drawing it! 😊

49

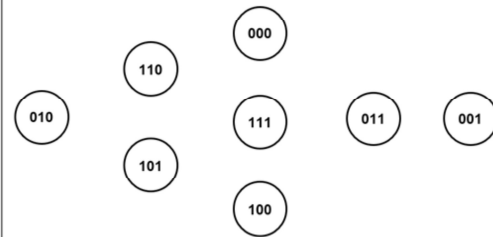
Now, let's start drawing the state diagram!



Analysis Step 3: Draw the state diagram

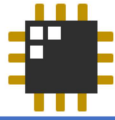
Input	Current State			Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	1	1	1	0	1	1	0
0	0	0	1	0	1	1	1	0	1	1	1
0	0	1	0	0	1	1	0	0	0	0	0
0	0	1	1	0	1	1	0	0	0	0	1
0	1	0	0	0	1	0	1	1	1	1	0
0	1	0	1	0	1	0	1	1	1	1	1
0	1	1	0	0	1	0	0	1	1	0	0
0	1	1	1	0	1	0	0	1	1	0	1
1	0	0	0	0	1	1	1	0	1	1	0
1	0	0	1	0	1	1	1	0	1	1	1
1	0	1	0	1	1	1	0	1	0	0	0
1	0	1	1	1	1	1	0	1	0	0	1
1	1	0	0	0	1	0	1	1	1	1	0
1	1	0	1	0	1	0	1	1	1	1	1
1	1	1	0	1	1	1	0	0	0	0	0
1	1	1	1	1	1	1	0	0	0	0	1

Try drawing it! 😊



50

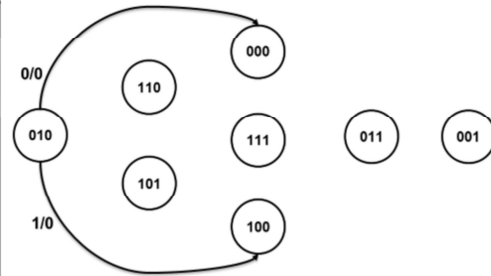
Once again, using codenames for the states is optional. We can simply represent each state using its corresponding combination of flip-flop outputs.



Analysis Step 3: Draw the state diagram

Input		Current State			Flip-flop Inputs				Next State			Output
X		A	B	C	T	J	K	D	A	B	C	Y
0		0	0	0	0	1	1	1	0	1	1	0
0		0	0	1	0	1	1	1	0	1	1	1
0		0	1	0	0	1	1	0	0	0	0	0
0		0	1	1	0	1	1	0	0	0	0	1
0		1	0	0	0	1	0	1	1	1	1	0
0		1	0	1	0	1	0	1	1	1	1	1
0		1	1	0	0	1	0	0	1	1	0	0
0		1	1	1	0	1	0	0	1	1	0	1
1		0	0	0	0	1	1	1	0	1	1	0
1		0	0	1	0	1	1	1	0	1	1	1
1		0	1	0	1	1	1	0	1	0	0	1
1		1	0	0	0	1	0	1	1	1	1	0
1		1	0	1	0	1	0	1	1	1	1	1
1		1	1	0	1	1	1	0	0	0	0	0
1		1	1	1	1	1	1	0	0	0	0	1

Try drawing it! 😊



If the output does not change with the input (only depends on the flipflop current states), you can put the output *inside* the state circle

- Remember, you don't leave the circle unless there is a trigger event.

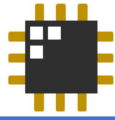
51

Let's examine state 010:

- When the input is 0, the state moves to 000, and the output remains 0. So, we draw an arrow from 010 to 000, labeled 0/0.
- When the input is 1, the state transitions to 100, with an output of 0. Thus, we draw an arrow from 010 to 100, labeled 1/0.

If the output remains constant regardless of the input, it can be placed inside the state circle.

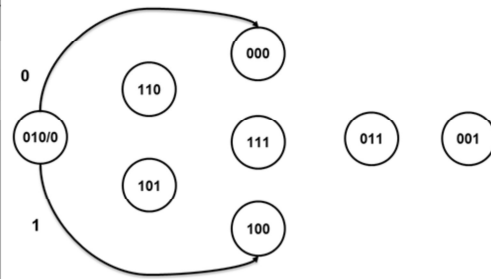
Note that we only transition to another state for every trigger event (i.e. positive edge of the clock). Thus, we can safely put the output inside the state if it does not change when the input changes.



Analysis Step 3: Draw the state diagram

Input		Current State			Flip-flop Inputs				Next State			Output
X		A	B	C	T	J	K	D	A	B	C	Y
0		0	0	0	0	1	1	1	0	1	1	0
0		0	0	1	0	1	1	1	0	1	1	1
0		0	1	0	0	1	1	0	0	0	0	0
0		0	1	1	0	1	1	0	0	0	0	1
0		1	0	0	0	1	0	1	1	1	1	0
0		1	0	1	0	1	0	1	1	1	1	1
0		1	1	0	0	1	0	0	1	1	0	0
0		1	1	1	0	1	0	0	1	1	0	1
1		0	0	0	0	1	1	1	0	1	1	0
1		0	0	1	0	1	1	1	0	1	1	1
1		0	1	0	1	1	1	0	1	0	0	1
1		1	0	0	0	1	0	1	1	1	1	0
1		1	0	1	0	1	0	1	1	1	1	1
1		1	1	0	1	1	1	0	0	0	0	0
1		1	1	1	1	1	1	0	0	0	0	1

Try drawing it! 😊



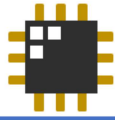
If the output does not change with the input (only depends on the flipflop current states), you can put the output *inside* the state circle

- Remember, you don't leave the circle unless there is a trigger event.

52

If the output remains unchanged regardless of the input, it can be placed inside the state circle, as shown in this figure.

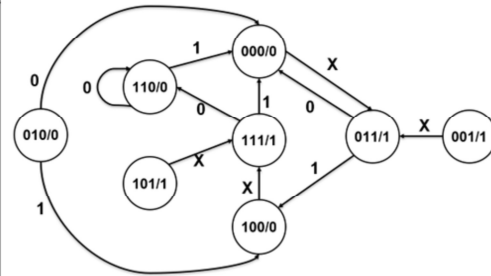
Observing the table, we see that the output stays the same for each state, so we can directly place the output to the corresponding state circles and label the arrows with only the input values.



Analysis Step 3: Draw the state diagram

Input		Current State			Flip-flop Inputs				Next State			Output
X		A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	0	1	1	1	0	1	1	0
0	0	0	0	1	0	1	1	1	0	1	1	1
0	0	0	1	0	0	1	1	0	0	0	0	0
0	0	0	1	1	0	1	1	0	0	0	0	1
0	1	0	0	0	0	1	0	1	1	1	1	0
0	1	0	0	1	0	1	0	1	1	1	1	1
0	1	1	0	0	0	1	0	0	1	1	0	0
0	1	1	1	1	0	1	0	0	1	1	0	1
1	0	0	0	0	0	1	1	1	0	1	1	0
1	0	0	0	1	0	1	1	1	0	1	1	1
1	0	1	0	0	1	1	1	0	1	0	0	0
1	0	1	1	1	1	1	1	0	1	0	0	1
1	1	0	0	0	0	1	0	1	1	1	1	0
1	1	0	0	1	0	1	0	1	1	1	1	1
1	1	1	0	0	1	1	1	0	0	0	0	0
1	1	1	1	0	1	1	1	0	0	0	0	0
1	1	1	1	1	1	1	1	0	0	0	0	1

Try drawing it! 😊



If the output does not change with the input (only depends on the flipflop current states), you can put the output *inside* the state circle

- Remember, you don't leave the circle unless there is a trigger event.

53

Once again, using our previously derived results, let's analyze state **010**:

- The output is 0, regardless of input.
- When the input is 0, the state transitions to **000** with an output of 0, so we draw an arrow from **010 to 000**, labeled 0.
- When the input is 1, the state moves to **100**, also with an output of 0, so we draw an arrow from **010 to 100**, labeled 1.

For state **000**:

- Regardless of the input, the state transitions to **011** with an output of 0. Since the output remains constant, we place **/0** inside the state circle and draw an arrow from **000 to 011**, labeled x (don't care).

For state **001**:

- Similar to **000**, the state always transitions to **011**, but with an output of 1. We place **/1** inside the state circle and draw an arrow from **001 to 011**, labeled x.

For state **011**:

- The output is 1, regardless of the input.

- When the input is **0**, the state moves to **000**, so we draw an arrow from **011 to 000**, labeled **0**.
- When the input is **1**, the state transitions to **100**, so we draw an arrow from **011 to 100**, labeled **1**.

For state **100**:

- The state always transitions to **111** with an output of **0**, so we place **/0** inside the state circle and draw an arrow from **100 to 111**, labeled **x**.

For state **101**:

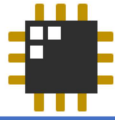
- The state always transitions to **111** with an output of **1**, so we place **/1** inside the state circle and draw an arrow from **101 to 111**, labeled **x**.

For state **110**:

- The output remains **0** regardless of the input.
- When the input is **0**, the state stays the same, so we draw a **self-loop** on **110**, labeled **0**.
- When the input is **1**, the state transitions to **000**, so we draw an arrow from **110 to 000**, labeled **1**.

For state **111**:

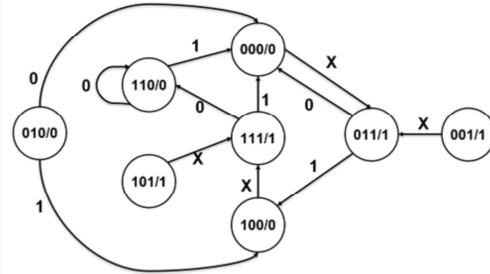
- The output is **1**, regardless of the input.
- When the input is **0**, the state moves to **110**, so we draw an arrow from **111 to 110**, labeled **0**.
- When the input is **1**, the state moves to **000**, so we draw an arrow from **111 to 000**, labeled **1**.



Analysis Step 3: Draw the state diagram

Input	Current State			Flip-flop Inputs				Next State			Output
X	A	B	C	T	J	K	D	A	B	C	Y
0	0	0	0	0	1	1	1	0	1	1	0
0	0	0	1	0	1	1	1	0	1	1	1
0	0	1	0	0	1	1	0	0	0	0	0
0	0	1	1	0	1	1	0	0	0	0	1
0	1	0	0	0	1	0	1	1	1	1	0
0	1	0	1	0	1	0	1	1	1	1	1
0	1	1	0	0	1	0	0	1	1	0	0
0	1	1	1	0	1	0	0	1	1	0	1
1	0	0	0	0	1	1	1	0	1	1	0
1	0	0	1	0	1	1	1	0	1	1	1
1	0	1	0	1	1	1	0	1	0	0	0
1	0	1	1	1	1	1	0	1	0	0	1
1	1	0	0	0	1	0	1	1	1	1	0
1	1	0	1	0	1	0	1	1	1	1	1
1	1	1	0	1	1	1	0	0	0	0	0
1	1	1	1	1	1	1	0	0	0	0	1

Try drawing it! 😊



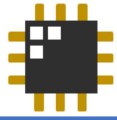
As you can see from the output equation, the input X is not present, thus the output is not directly dependent on the input.

- Output equations
- $Y = C$

54

Since the output equation $Y = C$ does not include the input X , this means that the output Y is solely determined by the state variable C and remains unaffected by changes in X .

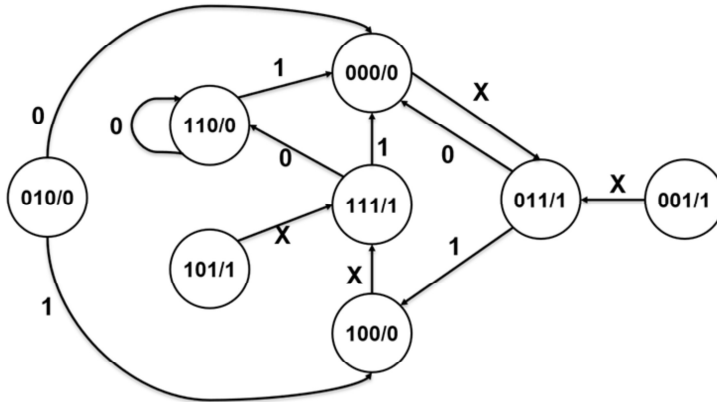
In other words, the system's output depends only on the current state and not directly on the input, making it a **Moore-type** output.



Analysis Step 3: Draw the state diagram

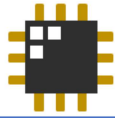
Note that you only go from one circle to another on every trigger event!

- Changing the input simply changes the arrow you're taking, but transferring circles needs the trigger (e.g. positive edge of the clock) to happen every time

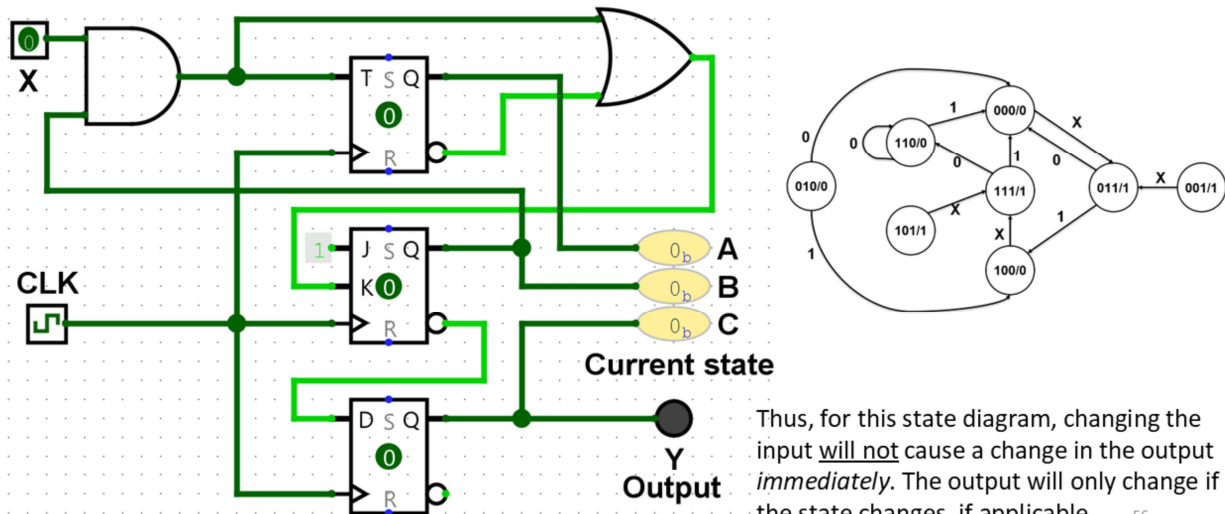


Thus, for this state diagram, changing the input will not cause a change in the output *immediately*. The output will only change if the state changes, if applicable. 55

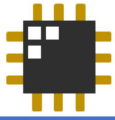
As mentioned earlier, state transitions only occur on a trigger event, such as the positive edge of the clock. While the input determines which transition will happen, the system remains in its current state until the trigger event occurs. Thus, if the current output remains the same regardless of the input, the output can be written inside the state itself. This means that the output only depends on what the state is, and not the input.



Analysis Step 3: Draw the state diagram

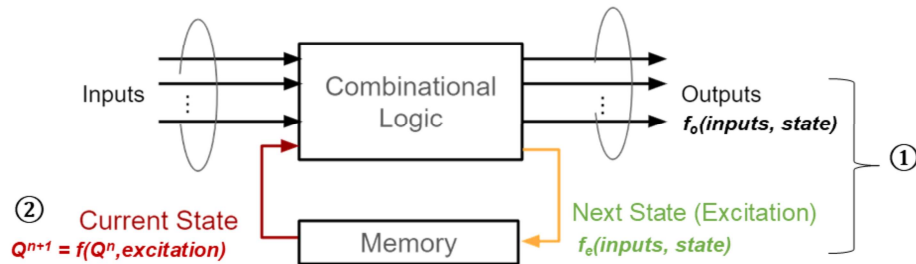


For a clearer understanding, you can refer to the GIF shown on this slide.



Summary: Analysis of Sequential Circuits

Given a circuit, we want to know what it does → state diagram*



Using the model for analysis, we can obtain all the information needed by

- ① inspecting the combinational logic block and
- ② knowing how the memory elements behave (characteristic tables) to obtain
- ③ a state diagram

57

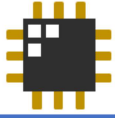
In summary, we started by analyzing the given circuit and deriving the excitation and output equations.

Using these equations, we constructed the state transition table by determining the next states of the flip-flops based on their characteristic tables.

We then mapped these states into a state transition diagram, ensuring each state transition was correctly represented by arrows labeled with input/output values.

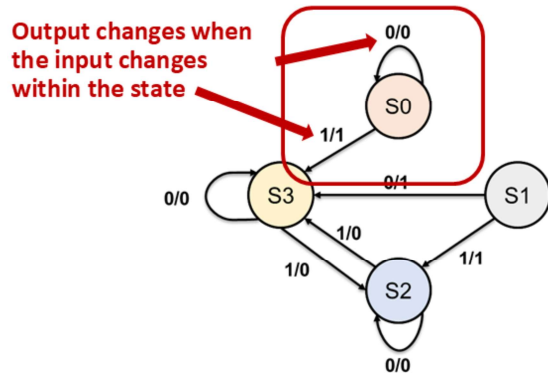
Additionally, we discussed how outputs that remain constant regardless of input can be placed inside the state circles.

Finally, we emphasized that state transitions only occur on trigger events.

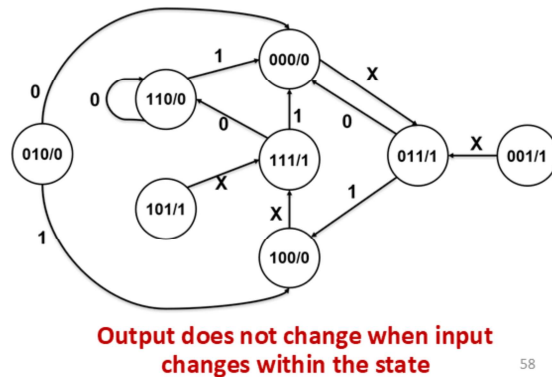


Segue: Mealy vs Moore machines

Mealy machines are sequential circuits where the output depends on the input and the current state

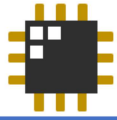


Moore machines are sequential circuits where the output only depends on the current state

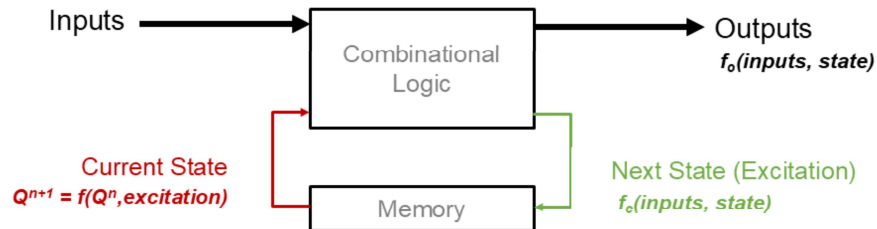


58

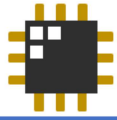
Mealy and Moore machines are two types of finite state machines (FSMs) used in sequential circuit design. The key difference lies in how they determine outputs.



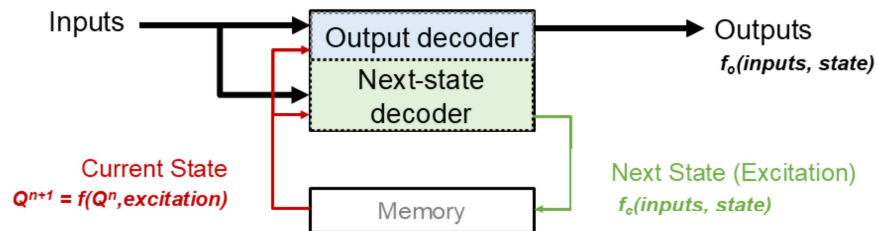
Recall: Canonical model



The **canonical model for sequential circuits** provides a structured way to represent state-based systems using state variables, input variables, and output functions. The circuit's behavior can be represented using a **state transition table** or a **state diagram**, helping in design, analysis, and optimization. This model is essential for developing efficient and predictable sequential circuits in digital systems.



Output and next-state equations



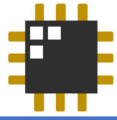
Output decoder = output equations from the circuit

Next-state decoder = flipflop excitation equations of the circuit

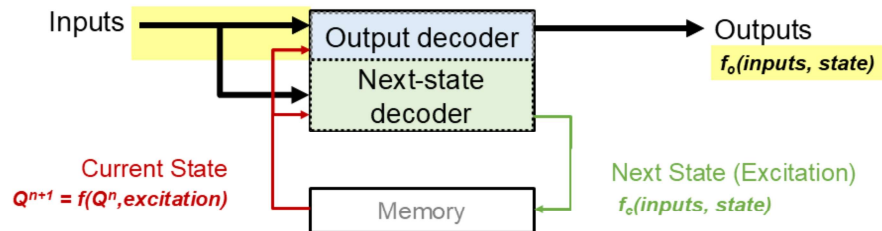
In a sequential circuit, the **next-state decoder** determines the next state based on the current state and inputs, while the **output decoder** generates the output based on the current state or both the current state and inputs, ensuring proper state transitions and output behavior.

The **output decoder** is essentially the output equation that you derive from the circuit.

The **next-state decoder** is essentially the flipflop excitation equations from the circuit!

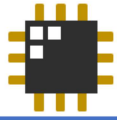


Mealy and Moore Machines

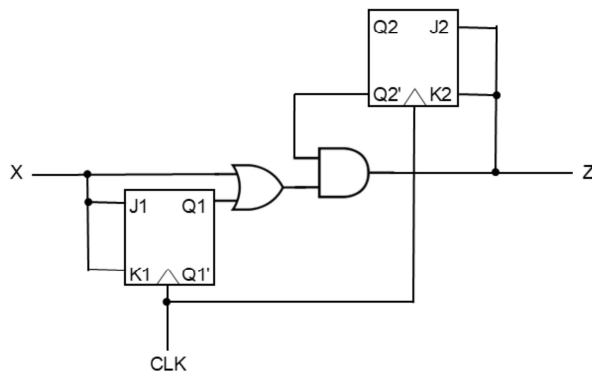


If outputs are a function of both the present state and the inputs, the machine is a **Mealy machine**.

A **Mealy machine** generates outputs based on both the current state and the input. This means the output can change immediately when the input changes, even within the same state. As a result, Mealy machines tend to react faster but may introduce glitches if inputs are not stable.



Example: Mealy machine



Excitation equations

- $J1 = K1 = X$
- $J2 = K2 = Q2'(X + Q1)$

Output equations

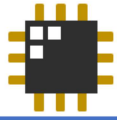
- $Z = Q2'(X + Q1)$

the output depends on the input

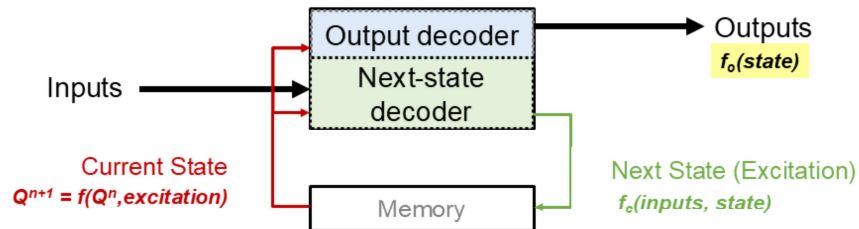
For example, consider the output equation of this circuit:

$$Z = Q2' (X + Q1)$$

Notice that the equation includes X, which is an input. Since the output depends on both the current state and the input, we can confirm that this system is a Mealy machine.

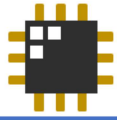


Mealy and Moore Machines

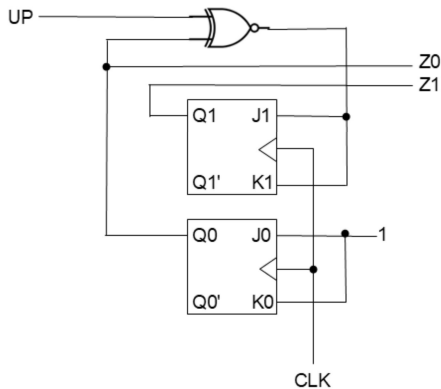


If outputs are a function of the present state only,
the machine is a **Moore machine**.

On the other hand, a **Moore machine** determines outputs solely based on the current state, making the output independent of the input. This ensures more stable outputs but may introduce a slight delay, as outputs only update on state transitions.



Example: Moore machine



Excitation equations

- $J1 = K1 = (UP \oplus Q0)'$
- $J0 = K0 = 1$

Output equations

- $Z1 = Q1$
- $Z0 = Q0$

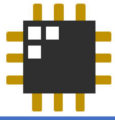
outputs do NOT depend on input,
only on state

For this example, let's consider the output equations:

$$Z1 = Q1$$

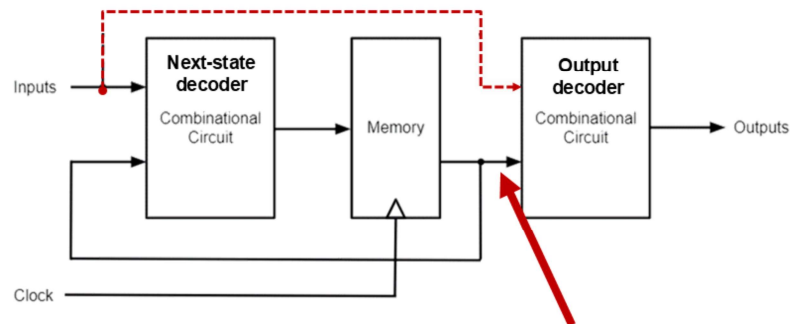
$$Z0 = Q0$$

Since the equations do not include **UP** (the input), the outputs depend only on the current state. This confirms that the system is a **Moore machine**.

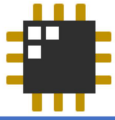


Mealy and Moore Machines

Moore machines do NOT have this 'Mealy path'

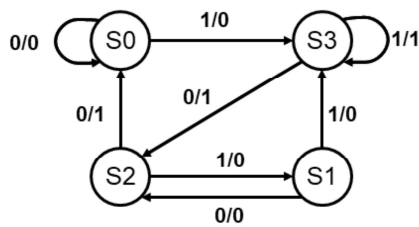


In a Moore machine, there's no such thing as a "Mealy path." This refers to how Mealy machines allow outputs to change immediately in response to input changes, even within the same clock cycle. Since Mealy machines depend on both the current state and the input, any variation in the input can instantly affect the output. However, in Moore machines, outputs are strictly determined by the current state alone. This means the output remains stable throughout the clock cycle, only updating when the system transitions to a new state on a clock edge. This difference makes Moore machines more predictable, while Mealy machines can react faster but may introduce glitches if the input fluctuates unexpectedly.



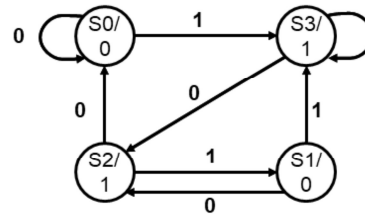
State diagram conventions

- **Mealy** machine has both the input and output values indicated along the arrows between the circles. (i.e. INPUT/OUTPUT)



Input/Output labels per transition

- **Moore** Machine has only the inputs indicated along the arrows. Output values associated with each state are indicated inside the circles.



Input label per transition, output label per state

Moore and Mealy machines are two types of finite state machines (FSMs) used in sequential circuits. In a **Moore machine**, the output depends only on the current state, making it more stable but slightly slower in response. In contrast, a **Mealy machine** determines its output based on both the current state and input, allowing for faster responses but with potential glitches. Moore machines follow the equation $Z = f(Q)$, while Mealy machines use $Z = f(Q, X)$. In state diagrams, **Moore machines label outputs inside the state circles**, while **Mealy machines label outputs on the transitions (arrows)** since the output changes dynamically with the input. Generally, Moore machines are preferred for stable outputs, while Mealy machines are used when quick reactions to inputs are needed.